CollabSphere

A Collaboration Marketplace for Creators, Influencers, and Brands

Repository	https://github.com/samiksha-jangid27/CollabSphere
Frontend	Next.js App Router frontend with reusable components, hooks, contexts, and service wrappers
Backend	Express + TypeScript API with modular controllers, services, repositories, and middleware
Data Layer	MongoDB with Mongoose schemas, indexes, and references
Core Flow	Authentication, profile management, discovery/search, collaboration requests, and messaging support
Design Focus	Layered architecture, SOLID principles, reusable patterns, and maintainable module boundaries

This report covers: folder structure, file roles, architecture explanation, OOP concepts, design patterns, SOLID principles, database design, tests, contribution matrix, and appendix diagrams.

Prepared by	Sarvesh Srinath, Arina Ali, Samiksha Jangid, Saksham Sharma, Yachna
Source Basis	Code-first review of the repository and tracked project documentation

Contents

1. Project Overview

CollabSphere is a full-stack collaboration marketplace designed to connect creators, influencers, and brands in one organized platform. The repository is split into a frontend application, a backend API, documentation, and UML sources. The codebase emphasizes a clear separation between presentation, application logic, persistence, and shared utilities.

Key idea: the application is built as a layered system. The frontend handles UI and state, the backend handles business logic and security checks, and MongoDB stores the persistent data. Shared modules keep the codebase consistent across features.

2. Folder Structure Analysis

2.1 Root

The root contains repository-level configuration, documentation, and architecture material.

- `client/` — frontend application.
- `server/` — backend API and tests.
- `docs/` — written specifications and roadmap material.
- `UML Designs/` — system design diagrams and Mermaid sources.
- `CLAUDE.md` — development guardrails and coding conventions.
- `README.md` — project summary and diagram index.

2.2 Client

The frontend is organized around Next.js App Router conventions.

- `app/` — route groups and pages for auth and the main app views.
- `components/` — reusable UI by domain: profile, search, collaboration, messaging, and primitive UI.
- `context/` — global state providers such as auth and socket state.
- `hooks/` — custom hooks that wrap stateful logic and API interaction.
- `services/` — API service wrappers and transport setup.
- `types/` — TypeScript contracts for data exchanged with the backend.

2.3 Server

The backend is organized as a feature-based Express application.

- `config/` — environment, database, cloud, and socket initialization.

- `middleware/` — authentication, authorization, validation, uploads, rate limiting, and error handling.
- `models/` — Mongoose schemas for users, profiles, collaboration, conversations, and messages.
- `modules/` — domain slices with controller, service, repository, route, validation, and interface files.
- `shared/` — reusable base repository, event bus, constants, errors, logger, and response helper.
- `tests/` — unit and integration tests organized by module.

3. File-by-File Analysis

3.1 Root-Level Files

File	Role
`.nvmrc`	Pins the Node runtime so all environments use the same version.
`.gitignore`	Excludes generated artifacts, local environment files, and dependencies from version control.
`package.json`	Orchestrates root-level scripts for running the monorepo.
`CLAUDE.md`	Stores repository conventions, architectural constraints, and implementation guidance.
`README.md`	Presents the project summary and links the tracked diagram sources.
`SPRINT_4_IMPLEMENTATI`	Captures milestone notes and delivery decisions.
`skills-lock.json`	Tooling metadata used by the environment.

3.2 Client Files

File Area	What It Does
`client/src/app`	Defines application routes, layouts, and route groups for authentication and the main workspace.
`client/src/components`	Contains reusable primitives such as buttons, inputs, cards, and OTP controls.
`client/src/components`	Holds profile-related UI for editing, viewing, and avatar management.
`client/src/components`	Implements discovery UI such as search bars, filters, and profile grids.
`client/src/components`	Supports collaboration request creation, listing, and card display.
`client/src/context`	Manages shared application state such as authentication and socket connectivity.
`client/src/hooks`	Encapsulates reusable client-side logic and asynchronous state.
`client/src/services`	Centralizes HTTP calls and token-aware API access.
`client/src/types`	Defines shared TypeScript contracts used by the frontend.

3.3 Server Files

File Area	What It Does
`server/src/index.ts`	Boots the Express application, registers middleware, mounts routes, and starts the server.
`server/src/config`	Initializes environment variables, database connectivity, and socket setup.
`server/src/middleware`	Applies request-level concerns such as auth, roles, validation, and errors.
`server/src/models`	Stores the MongoDB schemas that represent the domain entities.
`server/src/modules`	Implements each feature as a self-contained slice with controller, service, repository, and route files.
`server/src/shared`	Provides reusable abstractions and cross-cutting utilities.
`server/tests`	Verifies the behavior of the backend with isolated and integration-style tests.

4. System Design Explanation

The application follows a client-server architecture. The frontend is responsible for rendering pages, collecting input, and calling backend services. The backend validates requests, applies authorization rules, executes business logic, and persists data in MongoDB.

4.1 Main Flow

- The user interacts with a frontend page or component.
- A service wrapper sends the request to the backend.
- Middleware verifies authentication, authorization, and input shape.
- A controller delegates to the relevant service.
- The service uses a repository or model to read and write data.
- Shared helpers format the response consistently.

4.2 System Design Value

- **Scalability:** feature-based modules make it easier to extend the system without rewriting core layers.
- **Maintainability:** controllers stay thin, services stay focused, and repositories isolate persistence logic.
- **Testability:** small modules and clear boundaries support mocking and isolated tests.
- **Consistency:** shared helpers and base abstractions reduce duplicated logic.

5. Architecture Overview

Layered structure: Frontend components → API services → backend controllers → services → repositories/models → MongoDB. This separation makes the code easier to understand and refactor.

Frontend	Route-based pages, reusable UI components, contexts, hooks, and service wrappers.
Backend API	Express routes, middleware, controllers, and feature services.
Persistence	Mongoose models and MongoDB collections.
Shared Core	Base repository, event bus, logger, errors, constants, and response helper.
External Services	Email, cloud storage, authentication, and real-time support.

6. OOP Concepts Used

Concept	Where It Appears	Why It Matters
Encapsulation	Models, services, context providers, and reusable components	Keeps related state and behavior together.
Inheritance	Base repository and custom error class	Reduces duplication and supports reuse.
Abstraction	Interfaces, service contracts, and shared helpers	Exposes only what the rest of the system needs.
Polymorphism	Interchangeable repositories, services, and handlers	Allows different implementations behind the same contract.
Composition	Modules built from smaller helpers, hooks, and UI pieces	Makes the system flexible and easier to extend.

7. Design Patterns Implemented

Pattern	Where It Appears	Purpose
Repository	Backend repositories	Separates database access from application logic.
Service Layer	Backend services	Holds the core business rules.
Middleware	Express middleware stack	Handles auth, roles, validation, uploads, and errors.
Observer / Event Bus	Shared event system	Decouples events from the code that reacts to them.
Facade	Response helper and API client wrappers	Simplifies frequently used operations.
Adapter	External service wrappers	Normalizes third-party APIs behind a stable interface.
Context / Provider	Frontend global state	Shares app state without prop drilling.
Custom Hook	Client hook files	Reuses stateful logic across components.

8. SOLID Principles

Principle	Reflection in the Repository
Single Responsibility	Controllers, services, repositories, middleware, and UI components each focus on one job.
Open / Closed	New features can be added by introducing new modules or components without changing the existing ones.
Liskov Substitution	Shared abstractions and base classes can be replaced with their specialized implementations.
Interface Segregation	Small, focused interfaces keep modules from depending on methods they do not need.
Dependency Inversion	Higher-level logic depends on abstractions and shared contracts instead of concrete low-level details.

9. Major Workflow

9.1 Collaboration Request Flow

A typical workflow begins when a user opens the discovery or collaboration screen and submits a request:

1. The frontend collects the request details and sends them through a service wrapper.
2. The backend validates the request and checks the authenticated user's role.
3. The controller passes the data to the collaboration service.
4. The service stores the request and updates related state in the database.
5. Shared mechanisms can notify other parts of the system if the request status changes.
6. The frontend reflects the updated request state after the response arrives.

9.2 Why This Flow Is Good

The flow keeps request handling predictable: the UI stays separate from the business logic, authorization remains centralized, and persistence logic remains inside repositories and models.

10. Database Design

User	Stores identity and role-related data.
Profile	Stores public information, creator details, and profile metadata.
Collaboration Request	Stores collaboration proposals and their status.
Conversation	Stores chat threads between users.
Message	Stores the individual messages exchanged in a conversation.

The model layer is designed to support the system's primary functions: account management, profile management, creator discovery, collaboration tracking, and communication.

11. Tests and Verification

The backend includes automated tests organized by module. The tests focus on the most important code paths: authentication, profile operations, search behavior, collaboration requests, and messaging support.

Area	What the Tests Check
Auth	Login and token-related behavior.
Profile	Profile creation, updates, and data handling.
Search	Filtering and discovery logic.
Collaboration	Request lifecycle and state changes.
Messaging	Conversation and message handling.

Testing note: the repository uses isolated backend tests with mocked or in-memory dependencies so that the main logic can be checked without needing a full production deployment.

12. Contribution Matrix

Team Member	Contribution Area
Sarvesh Srinath	Repository analysis & backend review
Arina Ali	Documentation & report coordination
Samiksha Jangid	Frontend structure & UI review
Saksham Sharma	Testing & validation review
Yachna	UML / architecture support

13. Conclusion

CollabSphere is organized around a clean full-stack architecture with clear responsibilities, reusable modules, and strong separation between the UI, API, and data layers. The repository structure supports both maintainability and future growth, while the use of design patterns, SOLID principles, and focused modules makes the codebase easier to understand and extend.

Appendix A: UML and ER Diagrams

The repository includes Mermaid sources for the system diagrams. Insert the exported, code-derived figures here if you want the report to show the visuals directly in the PDF.

Use Case Diagram	Diagram of actors and supported system actions.
Class Diagram	Diagram of controllers, services, models, and relationships.
Sequence Diagram	Diagram of the main request flow across frontend, backend, and database.
ER Diagram	Diagram of the data model and entity relationships.
Architecture Diagram	Diagram of the layered system and module boundaries.

Team Members

- Sarvesh Srinath
- Arina Ali
- Samiksha Jangid
- Saksham Sharma
- Yachna